Software requirements Specification (SRS)

MoodMatch app for mood journal and AI powered recommendations

Table of Contents

Table of Contents ii
Revision History ii

1. **Introduction** 1
   - **1.1 Purpose** 1
   - **1.2 Document Conventions** 1
   - **1.3 Intended Audience and Reading Suggestions** 1
   - **1.4 Product Scope** 1
   - **1.5 References** 1
2. **Overall Description** 2
   - **2.1 Product Perspective** 2
   - **2.2 Product Functions** 2
   - **2.3 User Classes and Characteristics** 2
   - **2.4 Operating Environment** 2
   - **2.5 Design and Implementation Constraints** 2
   - **2.6 Assumptions and Dependencies** 3
3. **External Interface Requirements** 3
   - **3.1 User Interfaces** 3
   - **3.2 Software Interfaces** 3
4. **System Features** 4
   - **4.1 User Authentication** 4
   - **4.2 Mood Capture** 5
   - **4.3 AI-Based Recommendations** 6
   - **4.4 Journal Management** 7
   - **4.5 Dashboard and Insights** 8
5. **Other Nonfunctional Requirements** 9
   - **5.1 Performance Requirements** 9
   - **5.3 Security Requirements** 9
   - **5.4 Software Quality Attributes** 10
6. **Other Requirements** 11

Appendix A: Glossary 12

1. Introduction

1.1 Purpose

The purpose of the SRS for MoodMatch is to clearly define the functional and non-functional requirements of the application, including mood detection, AI-based recommendations, and user data management. It serves as a structured guide for development, testing, and future enhancements, ensuring all stakeholders share a common understanding of the system.

1.2 Document Conventions

This document follows a standard Software Requirements Specification (SRS) structure. Technical terms, abbreviations, and naming conventions are used consistently throughout to ensure clarity and alignment during development.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, project mentors, and evaluators involved in the design, implementation, and review of the MoodMatch application.

1.4 Product Scope

MoodMatch is an AI-powered web application that captures a user's emotional state through manual input or optional webcam-based detection and provides personalized recommendations such as music, activities, and reflections. The system also maintains a private mood journal with historical data, enabling users to track patterns over time while ensuring secure authentication and privacy of user data.

1.5 References

Commented [I1]: *Update references towards the end*

- Next.js Documentation – https://nextjs.org/docs
- Prisma Documentation – https://www.prisma.io/docs
- *face-api.js Documentation – https://justadudewhohacks.github.io/face-api.js/docs*

2. Overall Description

2.1 Product Perspective

MoodMatch is a standalone web application developed as a new system. It uses a frontend interface, backend services, a database for storing user data, and external AI APIs to generate personalized recommendations. The application also uses optional browser-based emotion detection, where webcam data is processed locally to maintain user privacy.

2.2 Product Functions

The main functions of MoodMatch include:

- User registration and login with secure authentication
- Capturing user mood through manual selection or optional webcam detection
- Generating personalized recommendations (music, activities, reflections) using AI
- Storing mood entries and recommendations in a private journal
- Viewing, editing, and deleting journal entries
- Displaying basic insights such as recent mood trends on a dashboard

2.3 User Classes and Characteristics

The primary users of MoodMatch are individuals who want to track their mood and receive personalized recommendations. These users are expected to have basic knowledge of web applications and will use features such as mood logging, viewing journal entries, and accessing AI-generated suggestions.

2.4 Operating Environment

MoodMatch operates as a web application accessible through modern web browsers such as Google Chrome, Microsoft Edge, or Safari. It runs on any system with internet access, including desktops and mobile devices. The application is built using Next.js for the frontend and backend, with a PostgreSQL database for data storage, and integrates external AI APIs for generating recommendations.

2.5 Design and Implementation Constraints

MoodMatch must operate as a web-based application accessible through modern browsers. Privacy constraints require that webcam-based emotion detection is performed entirely on the client side, without transmitting image data to the server. The system must also ensure secure handling of user data through proper validation and authentication mechanisms.

Additionally, the project follows a predefined technology stack including Next.js, TypeScript, and PostgreSQL, which guides the overall design and implementation.

2.6 Assumptions and Dependencies

MoodMatch assumes that users have access to a stable internet connection and a modern web browser for proper functionality. It also assumes that users grant necessary permissions for optional features such as webcam-based mood detection.

The system depends on external services such as AI APIs for generating recommendations, authentication services for secure login, and a PostgreSQL database for storing user data. Any disruption in these services may affect the performance or availability of certain features.

3. External Interface Requirements

3.1 User Interfaces

MoodMatch provides a web-based user interface designed for ease of use and accessibility across desktop and mobile devices. The interface includes pages such as login/signup, a dashboard, mood selection screen, journal view, and recommendations display. Users interact with the system through buttons, forms, and simple navigation elements.

Standard UI elements such as input fields, action buttons, and notifications are used consistently across the application. Error messages and confirmations are clearly displayed to guide user actions. The design follows a minimal and intuitive layout to ensure a smooth user experience.

3.2 Software Interfaces

MoodMatch interacts with several software components to function effectively. It connects to a PostgreSQL database through Prisma ORM for storing user data, journal entries, and recommendations. The system integrates with authentication services to manage user login and session handling.

It also communicates with external AI APIs to generate personalized recommendations based on user mood data. Additionally, browser-based libraries such as face-api.js are used for optional emotion detection on the client side. Data is exchanged between these components through API requests and responses.

4. System Features

4.1 User Authentication

4.1.1 Description and Priority

This feature allows users to register, log in, and securely access the MoodMatch application. It ensures that only authenticated users can access personal data such as mood journals and recommendations.

Priority: High

4.1.2 Stimulus/Response Sequences

• User registers → validated → account created → logged in
• User logs in → verified → dashboard access
• Invalid login → error shown
• Logout → session ends

4.1.3 Functional Requirements

- REQ-1: The system shall allow users to register using an email and password.
- REQ-2: The system shall validate user input during registration (e.g., email format, required fields).
- REQ-3: The system shall securely store user credentials using hashing techniques.
- REQ-4: The system shall allow registered users to log in using valid credentials.
- REQ-5: The system shall display an error message for invalid login credentials.
- REQ-6: The system shall create and manage user sessions upon successful authentication.
- REQ-7: The system shall restrict access to protected pages for unauthenticated users.
- REQ-8: The system shall allow users to log out and terminate their session

4.2 Mood Capture

4.2.1 Description and Priority

Allows users to capture mood manually or via webcam detection.

Priority: High

4.2.2 Stimulus/Response Sequences

• Manual selection → mood recorded
• Webcam → detect → confirm mood
• Error → fallback to manual

4.2.3 Functional Requirements

- REQ-1: The system shall allow users to input their mood manually through a predefined set of options.
- REQ-2: The system shall provide an option to detect the user's mood using webcam-based emotion recognition.
- REQ-3: The system shall process webcam data entirely on the client side without sending image data to the server.
- REQ-4: The system shall display the detected mood and require user confirmation before saving.
- REQ-5: The system shall handle errors such as denied camera permission or no face detected and prompt the user to use manual input.

4.3 AI-Based Recommendations

4.3.1 Description and Priority

Generates personalized recommendations based on mood and history.

Priority: High

4.3.2 Stimulus/Response Sequences

• Mood confirmed → AI called → results shown
• API failure → error message

4.3.3 Functional Requirements

- REQ-1: The system shall generate personalized recommendations based on the user's current mood.
- REQ-2: The system shall use the user's recent journal entries to enhance recommendation relevance.
- REQ-3: The system shall provide recommendations in multiple categories such as music, activities, and reflections.
- REQ-4: The system shall display the generated recommendations to the user after processing.
- REQ-5: The system shall handle failures in AI API responses and notify the user appropriately.

4.4 Journal Management

4.4.1 Description and Priority

Stores and manages mood entries and recommendations.

Priority: High

4.4.2 Stimulus/Response Sequences

• Mood logged → entry created
• View journal → entries shown
• Edit/delete → updated

4.4.3 Functional Requirements

- REQ-1: The system shall store each mood entry along with a timestamp and associated recommendations.
- REQ-2: The system shall allow users to view a list of their past journal entries.
- REQ-3: The system shall allow users to view details of individual journal entries.
- REQ-4: The system shall allow users to edit notes associated with a journal entry.
- REQ-5: The system shall allow users to delete journal entries.

4.5 Dashboard and Insights

4.5.1 Description and Priority

Displays mood trends and summaries.

Priority: Medium

4.5.2 Stimulus/Response Sequences

• Open dashboard → data shown
• No data → empty state

4.5.3 Functional Requirements

- REQ-1: The system shall display recent mood trends based on the user's journal data.
- REQ-2: The system shall provide a summary of user activity, such as total entries and common moods.
- REQ-3: The system shall update dashboard data dynamically when new entries are added.
- REQ-4: The system shall display an appropriate message when no data is available.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system should respond to user actions such as login, mood submission, and navigation within **2–3 seconds** under normal conditions. AI-generated recommendations may take slightly longer but should be displayed within **5–8 seconds**. The application should handle multiple users efficiently without noticeable performance degradation.

5.3 Security Requirements

The system must ensure secure authentication and protect user data at all times. User passwords must be stored using hashing techniques, and access to user data must be restricted based on authentication.

Webcam-based emotion detection must be processed entirely on the client side, ensuring that no image data is sent to the server. All inputs must be validated to prevent unauthorized access or data manipulation. Secure communication (HTTPS) must be used for all data transfers.

5.4 Software Quality Attributes

- Usability: The system should provide a simple and intuitive interface for all users.
- Reliability: The system should function consistently with minimal errors during normal usage.
- Performance: The application should maintain fast response times for core features.
- Maintainability: The codebase should be modular and easy to update or extend.
- Scalability: The system should support an increasing number of users without major changes.

6. Other Requirements

The system must maintain a structured database for storing user information, journal entries, and recommendations. Data should be organized efficiently to support quick retrieval and updates.

The application should be designed to support future enhancements such as additional recommendation types or advanced analytics. Proper documentation and clean coding practices should be followed to ensure ease of future development.

Appendix A: Glossary

1. AI (Artificial Intelligence): Technology used to generate personalized recommendations based on user mood and data.
2. API (Application Programming Interface): A set of rules that allows different software components to communicate with each other.
3. Authentication: The process of verifying the identity of a user before granting access to the system.
4. Client-Side Processing: Execution of code on the user’s device (browser) instead of the server.
5. Database: A structured collection of data used to store user information, journal entries, and recommendations.

6. Frontend: The user interface of the application that users interact with directly.
7. Backend: The server-side logic that processes requests, handles data, and communicates with the database.
8. Prisma ORM: A tool used to interact with the PostgreSQL database using code instead of raw queries.
9. PostgreSQL: A relational database management system used for storing application data.
10. Session: A temporary interaction between the user and the system after successful login.
11. Webcam-Based Emotion Detection: A feature that analyzes facial expressions using the device camera to detect user mood.
12. Recommendation Engine: The system component that generates personalized suggestions based on user data.

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>